

VETRA — Site Documentation

This document explains what VETRA is, how the codebase is organized, what every page and feature does, how the pieces fit together, and what to do next if you want to turn this prototype into a production product. It covers the public marketing site, the buyer (customer) app, the vendor app, and the admin console.

1. What VETRA is

VETRA is a marketplace concept for buyers ("customers") and sellers ("vendors") in Nigeria, plus an internal **admin console** for VETRA staff to monitor and moderate the platform. Right now it is a **front-end prototype**: there is no server, no database, and no real authentication. Every page is a static HTML file styled with plain CSS and made interactive with small, dependency-free JavaScript files. Where a page needs to "remember" something between visits (currently only the admin console), it uses the browser's `localStorage` as a stand-in for a real database.

This matters for how you read the rest of this document: numbers like "₦1,284,300 revenue" or "58 orders" are **seed/demo data**, not real transactions. Buttons that say "Add Product," "Approve," or "Suspend" genuinely update the page and (in the admin console) persist across reloads — but only inside your own browser, not on a server other people can see.

2. Tech stack & conventions

- **No build step, no framework.** Plain HTML files, plain CSS files, plain `<script>` tags. Open any `.html` file directly in a browser and it works.
 - **One shared design system per app area.** `customer/assets/style.css`, `vendor/assets/style.css`, and `admin/assets/style.css` each define the *same* base design tokens and layout primitives (sidebar, header, bottom nav, stat cards, tables, modals, forms, switches). This is intentional, established duplication — each area's stylesheet is self-contained so a change in one app doesn't silently break another, at the cost of some repeated CSS between them. The root `style.css` is a separate, simpler design system used only by the public marketing pages (`index.html`, `about.html`, `contact.html`, `signin.html`, `signup.html`).
 - **"App shell" pattern.** Every logged-in page (customer, vendor, admin) uses the same skeleton: a collapsible left sidebar on desktop ($\geq 900\text{px}$), a bottom tab bar on mobile, a dark header with the VETRA logo, and a `<main class="page-shell">` content area. `assets/interactions.js` (customer, vendor) and `assets/ui.js` (admin) wire up the sidebar collapse/expand behavior identically in each area.
 - **Mock data lives in JS.** Product lists, orders, and customer/vendor records are hard-coded into the HTML or into a `assets/data.js` -style file, not fetched from an API.
 - **Live chat.** Every page includes the Smartsupp live-chat snippet, with `hideWidget: true` so it only opens when a page's "Support" icon calls `smartsupp('chat:open')` (see `assets/support.js`, shared verbatim between customer and vendor).
-

3. Folder map

```
VETRA/
├── index.html, about.html, contact.html      Public marketing site
├── signin.html, signup.html                  Buyer/Vendor auth (mock)
├── style.css, signup.css                     Public-site design system
├── main.js, signin.js, signup.js             Public-site + auth page JS
├──
├── customer/                                Buyer-facing app
│   ├── dashboard.html, explore.html, category.html,
│   │   store.html, cart.html, orders.html, chat.html,
```

```

├── notifications.html, settings.html
├── assets/ (style.css, interactions.js, filters.js, orders.js,
    │       report-issue.js, support.js)
├── vendor/                               Seller-facing app
│   ├── dashboard.html, products.html, orders.html,
│   │   earnings.html, profile.html, notifications.html
│   └── assets/ (style.css, interactions.js, product-actions.js,
│       │       add-product.js, orders.js, order-tracking.js,
│       │       reports.js, profile.js, kyc.js, payout.js, support.js)
├── extras/                               Retired/unused assets, kept but not referenced
│   └── logo.png, logo-mark.png, logo-mark-white.png, *-avatar-dummy.jpg
├── admin/                                NEW – staff-only console (this session)
│   ├── login.html                        Admin sign-in gate
│   ├── dashboard.html                    Platform overview
│   ├── customers.html                    Manage buyer accounts (list)
│   ├── customer-detail.html              One customer's full profile + activity
│   ├── vendors.html                      Manage vendor stores (list)
│   ├── vendor-detail.html                One vendor's full profile + activity
│   ├── reports.html                      Reports & disputes queue
│   ├── activity.html                     Audit trail / activity log
│   ├── settings.html                     Admin profile, team, platform switches
│   └── assets/ (style.css, data.js, ui.js, dashboard.js, customers.js,
│       │       customer-detail.js, vendors.js, vendor-detail.js,
│       │       reports.js, activity.js, settings.js)

```

4. Public site

Page	Purpose
index.html	Landing page — hero, featured categories/products, marketing content.
about.html	Brand story / mission.
contact.html	Contact info / form (static).
signin.html	Buyer/Vendor sign-in, toggled by a "Buyer / Vendor" tab. A small "Admin sign in" link at the bottom now routes staff to admin/login.html .
signup.html	Buyer/Vendor account creation, same toggle pattern as sign-in; supports a #Vendor URL hash to land directly on the vendor tab.

How sign-in "works" today: [signin.js](#) validates only that the visible fields aren't empty, then redirects — Buyer → [customer/dashboard.html](#) , Vendor → [vendor/dashboard.html](#) . No password is actually checked and no session is created. [admin/login.html](#) follows the same mock pattern, redirecting to [admin/dashboard.html](#) . **This is the single most important thing to replace before going live** — see §9.

[index.html](#)

- [.buy-now-btn](#) (4x, "Popular this week" product cards) — Each button carries [data-product](#) , [data-price](#) , [data-vendor](#) . Click opens the checkout modal via [openCheckout\(btn\)](#) in [script.js:92](#) . Populates [#checkout-vendor](#) , [#checkout-product-name](#) , [#checkout-product-price](#) , [#checkout-total-price](#) (price + flat [deliveryFee = 1500](#) , [script.js:75](#)), resets the 4 guest-detail fields, then unhides [#checkout-overlay](#) . State: none persisted, plain DOM.

- **#checkout-confirm-btn ("Confirm purchase")** — `script.js:125-154` . Validates all 4 fields (`checkout-name` , `checkout-phone` , `checkout-email` , `checkout-address`) are non-empty; email additionally checked against `emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/` (`script.js:90`). Invalid/empty fields get `style.borderColor = '#e0475c'` and focus jumps to the first bad field. On success: button text → "Processing...", disabled, then after a 700ms `setTimeout` swaps `#checkout-step-review` for `#checkout-step-success` and fills `#checkout-success-text` with an interpolated confirmation message. No real order is created (`// TODO: replace with a real checkout/payment API call.`).
- **#checkout-cancel-btn / #checkout-close-btn / clicking the overlay backdrop / Escape key** — all call `closeCheckout()` (`script.js:117`), which just sets `checkoutOverlay.hidden = true` .
- **.nav-toggle (mobile hamburger)** — `script.js:2-14` . Toggles `.open` class on `.nav-links` and flips `aria-expanded` ; clicking any nav link also closes the menu.
- **Smartsupp support widget** — not present on this page as a dedicated support button (index.html has no sidebar); the Smartsupp snippet loads but stays hidden until interacted with globally.

about.html

- No interactive JS-driven elements beyond the shared `.nav-toggle` mobile menu (`script.js:2-14`) — the rest of the page is static marketing content. `script.js` 's FAQ-accordion and checkout-modal code look for elements (`.faq-item` , `#checkout-overlay`) that don't exist on this page, so those code paths simply no-op (`if (!q || !a) return;` etc.).

contact.html

- **.audience-toggle button[data-mode] ("I'm a buyer" / "I'm a Vendor" tabs)** — `script.js:35-48` . Click toggles `.active` on the clicked button, then shows/hides `#buyer-fields` vs `#Vendor-fields` by setting `style.display = 'grid' / 'none'` . Pure DOM toggle, no validation change (both field groups' inputs are optional; only `#name` , `#email` , `#message` on the base form carry `required`).
- **#contact-form submit ("Send message" button)** — `script.js:51-61` . `e.preventDefault()` , swaps the submit button's text to "Message sent", calls `form.reset()` , then restores the original button text after a `2600 ms` `setTimeout` . No network request — the reset happens immediately, so the "sent" state is purely cosmetic feedback and the form fields are already cleared before the label reverts.
- **.faq-item .faq-q (4 FAQ questions)** — `script.js:17-32` . Click toggles `.open` on the clicked `.faq-item` ; closes any other currently-open item first (single-open accordion), and animates via `el.querySelector('.faq-a').style.maxHeight = a.scrollHeight + 'px'` (or `null` when closing).

signin.html

- **.toggle button[data-mode] (Buyer / Vendor tabs)** — `signin.js:8-30` . Toggles `active` / `aria-selected` on buttons, shows/hides `#buyer-form` vs `#Vendor-form` via `.hidden` class, updates `.subtitle` copy, and hides `#guest-continue-wrap` when on the Vendor tab (guest checkout is buyer-only). Also runs on page load if URL has `#Vendor` hash (`signin.js:34-37`), auto-clicking the Vendor tab button — this is how `signup.html` 's post-registration redirect and `index.html` 's "Become a Vendor" links land users on the right tab.
- **.continue-btn ("Continue")** — `signin.js:50-82` . Reads whichever form (`buyer-form` 's `#username` / `#password` , or `Vendor-form` 's `#store-username` / `#Vendor-password`) matches the active tab. Validation: both fields must be non-empty (no format/regex check) — empty fields get `borderColor = '#e0475c'` . On success: button text → "Signing in...", disabled, then after `250ms` redirects via `window.location.href` to `DASHBOARD_PATHS[activeMode]` (`signin.js:45-48`): `customer/dashboard.html` for buyer, `vendor/dashboard.html` for Vendor. No real auth — this is purely a mock gate.
- **.google-btn ("Continue with Google")** — no JS handler attached anywhere; clicking does nothing (dead button, not wired).
- **#guest-continue-link ("Continue as Guest →")** — plain `<a href="customer/dashboard.html">` , no JS — a direct navigation link (buyer tab only; the whole `#guest-continue-wrap` is hidden on the Vendor tab).
- **Password show/hide toggle** — injected by `password-toggle.js:18-50` on every `input[type=password]` on the page (so both `#password` and `#Vendor-password`). Wraps the input in `.password-field-wrap` , appends a `.password-toggle-btn` ; click flips `input.type` between `password` / `text` and swaps the eye/eye-slash SVG icon. Local DOM state only (`input.type`), resets on reload.

signup.html

- `.toggle button[data-mode]` (Buyer / Vendor tabs) — `signup.js:7-31` via `activateMode(mode)` . Same pattern as `signin.js`: toggles active state, shows/hides `#buyer-form` / `#Vendor-form` , updates `.subtitle` . Runs on load if `location.hash === '#Vendor'` (`signup.js:28-31`) — this is the entry point for `index.html` 's `signup.html#Vendor` links and `vendor-protection.html` 's "Become a Vendor" CTA.
- `.continue-btn` ("Continue") — `signup.js:33-62` . Validates every `input` inside whichever form is active is non-empty (`requiredFields.forEach` , checks `field.value.trim()`); the enclosing `.input-wrap` gets `style.borderColor = '#e0475c'` per empty field, cleared to `''` when valid — no other validation (no email-format or phone-length regex despite `type="email"` / `type="tel"` inputs). On success: button text → "Creating account...", disabled, then after `400ms` redirects to `signin.html` (buyer) or `signin.html#Vendor` (Vendor) via `window.location.href` (`signup.js:58-60`). No account is actually created anywhere.
- `.google-btn` — unwired, same as `signin.html`.
- **Password toggle** on `#password` and `#Vendor-password` — same `password-toggle.js` behavior as `signin.html`.

`buyer-protection.html` / `vendor-protection.html`

- No page-specific JS interactions — purely static informational/marketing content (hero, value-grid cards, step timelines, CTA links). Only the shared `script.js` mobile-nav toggle (`.nav-toggle`) applies, and neither page has the FAQ/checkout/contact-toggle elements those code paths target, so those parts of `script.js` no-op. All links (`Start shopping` , `Become a Vendor` , `Talk to support` , etc.) are plain anchor navigation, not JS-driven.

5. Customer (buyer) app

All pages live in `customer/` and share the sidebar: **Dashboard, Explore, Categories, Cart, Chat, Notifications, Settings** (via the header/profile icon).

Page	What it does
<code>dashboard.html</code>	Home feed: promo banner carousel, category rail, featured product grid.
<code>explore.html</code>	Full product browse grid.
<code>category.html</code>	Category grid + a filtered product view; <code>assets/filters.js</code> (<code>initProductFilters</code>) filters the visible product cards client-side by category/price and shows an empty state if nothing matches.
<code>store.html</code>	A single vendor's storefront (name, products, and a Reviews section — see below) — <code>id="store-name"</code> is filled in by JS from a query param/mock lookup.
<code>cart.html</code>	Cart line items, quantity steppers, delivery-option radios, and an order summary card with a "Checkout" button.
<code>orders.html</code>	New — order history and per-order delivery tracking. Status filter tabs (All/Pending/Processing/Shipped/Out for delivery/Delivered/Cancelled) via <code>wireOrderFilterTabs</code> ; each order card is also a click-to-expand accordion (<code>wireOrderExpand</code> , both in <code>assets/orders.js</code>) revealing a step timeline (Placed → Processing → Shipped → Out for delivery → Delivered) that mirrors the escrow flow described on <code>../buyer-protection.html</code> . Each order's Report an issue button opens a real "File a report" modal (see below) rather than just opening live chat. Not in the main sidebar/bottom-nav (both are already at their intended item count) — reached from Settings → Account overview → "View orders" and from the "Order delivered" notification card on <code>notifications.html</code> .
<code>chat.html</code>	Buyer↔vendor messaging UI — conversation list + message thread, styled like a chat app (not connected to a real messaging backend).
<code>notifications.html</code>	List of notification cards (order updates, promos); the "Order delivered" card links to <code>orders.html</code> .
<code>settings.html</code>	Account & preference toggles (notifications, etc.); the "Orders placed" row under Account overview links to <code>orders.html</code> .

`customer/assets/interactions.js` wires: sidebar collapse/reopen, the banner carousel's dots, and `wireAddToCartButtons` (adds a fake line item / visual feedback when "Add to cart" is clicked — there's no real cart persistence between pages, since nothing here uses `localStorage`).

Reviews (`store.html`). Each mock vendor in the page's `VENDORS` object now carries a `reviews` array (name, rating, date, text), rendered as an average-score summary plus a review-card list, both tagged "Verified purchase" per the "verified reviews only" claim made elsewhere on the site. A "Write a review" form (star-picker + textarea) prepends a new review in-memory on submit — since there's no backend, it can't actually check for a completed order, so the form's hint text says as much and every submission is treated as if that check passed.

File a report (`orders.html` , `customer/assets/report-issue.js`). Each order's "Report an issue" button now opens a real modal — a reason dropdown, a details textarea, and a note pointing to `../buyer-protection.html` 's escrow-hold explanation — instead of just opening live chat. This is the one place in the customer app that breaks the "customer pages never touch admin's `localStorage` " rule: `orders.html` loads `../admin/assets/data.js` directly and calls its new `VetraAdmin.addReport()` function, so a report filed here actually lands in the same `vetra_admin_state_v1` state `admin/reports.html` reads — tested and confirmed to show up there, attributed to the (mock) signed-in buyer "Amaka Obi," with no false "handled by an admin" attribution on the activity-log entry (`logActivity()` gained a `systemEvent` option specifically so a buyer-originated event doesn't get credited to whichever admin last signed in on that browser). Each order card carries a `data-vendor-id` matching a real `admin/assets/data.js` vendor id where one exists (`Naija Home Essentials` → `v3`) and a synthetic one otherwise (the customer app's own separate `store.html` vendor directory was never unified with admin's vendor list — see §8) — either way the report is created and queued correctly, it just won't deep-link to a real vendor-detail page for the synthetic ids. Submitting swaps the button for a "Report filed" tag so the same order can't be reported twice.

Shared JS loaded on nearly every page: `assets/interactions.js` (sidebar collapse/reopen, banner carousel, global add-to-cart delegation) and `assets/support.js` (Smartsupp live-chat open/close). Both are IIFEs/top-level listeners bound on `DOMContentLoaded` .

Shared interaction reference (applies to every customer page unless noted): - `#header-sidebar-toggle` — `Vetra.wireSidebarToggle()` (`customer/assets/interactions.js:18-37`). Toggles `.hidden-desktop` on `#app-sidebar` , sets inline `display: none / ''` , and shows `#app-sidebar-reopen` (adds `.show`) when hidden. - `#app-sidebar-reopen` (floating ≡ button) — same function, `interactions.js:30-36` ; un-hides the sidebar and hides itself. - `#sidebar-collapse-toggle` (chevron inside sidebar) — `Vetra.wireSidebarCollapse()` (`interactions.js:39-47`); toggles `.collapsed` on `#app-sidebar` (icon-only rail mode). - `.support-btn` (sidebar "Support" link + bottom-nav "Support" item) — global delegated click listener in `customer/assets/support.js:75-80` calls `openLiveChat()` (`support.js:68-73`), which adds `body.livechat-open` , calls `smartsupp('chat:open')` , and lazily injects a `#livechat-close-btn` (`ensureCloseButton()` , `support.js:44-66`) whose click calls `smartsupp('chat:close')` then `window.location.reload()` . - `.add-cart / .add-btn` (any "Add to Cart" button on any product card, delegated at `document` level) — `Vetra.wireAddToCartButtons()` (`interactions.js:75-81`); simply navigates to `cart.html` . No item data is actually passed/stored — there is no real cart persistence anywhere in this app (confirmed: no `localStorage` usage in any customer JS file).

`customer/dashboard.html`

- **Banner carousel dots (`.banner-wrap .dots span`)** — `Vetra.wireBannerCarousel()` (`interactions.js:49-70`). Click on a dot calls `goTo(index)` , swapping `.active` between `.banner-slide` elements and their matching dot; also auto-advances every `4000ms` via `setInterval` when there's more than one slide. State: local `current` variable inside the closure, resets on reload. (Note: `dashboard.html` as shipped only has a single `#app-banner` section without the multi-slide markup used on `explore.html` — the carousel function still runs but only matters where 2+ `.banner-slide` elements exist.)
- **"See more" (`.cat-head .see-more`)** — plain `<button>` , no `id` /handler anywhere in the codebase; currently inert (does nothing on click).
- **`.add-cart` buttons (6 product cards)** — see shared reference above; routes to `cart.html` .
- **Header icon links (`notifications.html` , `cart.html` , `settings.html` via avatar)** — plain `<a>` navigation, not JS.

`customer/explore.html`

- **Filter button `#exploreFilterBtn`** — `initProductFilters()` in `customer/assets/filters.js` , invoked with `{ triggerSelector: '#exploreFilterBtn', panelId: 'exploreFilterPanel', gridSelector: '#app-products .grid' }` (inline script at the bottom of the page). Click toggles `.open` on `#exploreFilterPanel` (`filters.js:78-80`); clicking anywhere outside the panel/button while open closes it (`filters.js:82-86` , delegated document click listener).

- **Filter panel controls** (`#exploreSortSelect` , category checkboxes `[data-filter-category]` , price checkboxes `[data-filter-price]`) — each `change` event runs `applyFilters()` (`filters.js:28-76`):
- Reads checked `[data-filter-category]` values and checked `[data-filter-price]` values (each price checkbox's `value` is a "min-max" string, e.g. `"0-20000"` , `"150000-0"` where `0` upper bound means unbounded — `filters.js:44-50`).
- A card matches if `catMatch && priceMatch` where `catMatch` requires `card.dataset.category` to be in the selected set (or no categories selected = match-all), and `priceMatch` requires `card.dataset.price` (parsed `Number`) to fall in at least one selected bucket.
- Non-matching cards get `style.display = 'none'` .
- Sorting: `sortValue` from `#exploreSortSelect` — `price-asc` / `price-desc` sort numerically on `data-price` ; `name-asc` uses `data-name.localeCompare` ; `featured` preserves original DOM order. Sorted cards are re-appended into the grid (`sorted.forEach((card) => grid.appendChild(card))`).
- Updates `filterBtn` 's `.active` class (non-zero selection count) and the `[data-filter-count]` text ("Showing X of Y" / "Showing all N").
- Creates a `.filter-empty-state` div once, lazily (`ensureEmptyState()` , `filters.js:19-26`), shown/hidden based on `visibleCount` .
- `[data-filter-clear]` ("Clear filters") — `filters.js:92-102` . Unchecks every checkbox, resets the sort `<select>` to `"featured"` , re-runs `applyFilters()` .
- **Banner dots** — same `Vetra.wireBannerCarousel()` as `dashboard.html` , but this page ships 2 real `.banner-slide` elements so autoplay is active.
- **.add-cart buttons (6 products + featured-category cards)** — shared add-to-cart → `cart.html` .
- **Vendor mini-cards** (`.vendor-mini-card` , e.g. `store.html?vendor=gods-favour`) — plain anchor links to `store.html` , not JS-driven.
- **Trending chips** (`.trending-chip`) — plain `<a href="#">` , no handler.

customer/category.html

- **Filter button** `#categoryFilterBtn` — same `initProductFilters()` pattern as `explore.html` , configured with `gridSelector: '.products-grid'` and a slightly different filter panel (`Computing` / `Electronics` / `Phones & Tablets` categories, different price buckets: `0-5000` , `5000-15000` , `15000-25000`). Identical mechanics to `explore.html` 's filter (`filters.js`).
- **.add-cart buttons** — 6 product cards route to `cart.html` .
- **Category cards** (`.category-card`) — plain `<a href="#">` , not wired to any real per-category listing.
- **"See more" buttons** — inert, no handler.

customer/store.html

All logic lives in an inline `<script>` block at the bottom of the page (not a separate `.js` file). Vendor "database" is a hard-coded `VENDORS` object (`store.html:269-362`) keyed by `gods-favour` , `daniel-ice-fish` , `jennet-jeans` , `me-n-u-chops` .

- **Page load** — reads `?vendor=<id>` from the URL (`new URLSearchParams(window.location.search).get('vendor')`), looks it up in `VENDORS` , falling back to `VENDORS['gods-favour']` if missing/unknown (`store.html:367-369`). Populates `#store-avatar` , `#store-name` , `#store-bio` , `#store-rating` , `#store-orders` , `#store-response-time` , `#store-location` , `#store-member-since` , sets `#store-cover` 's `background-image` (vendor's `cover` image or `coverGradient` CSS string), sets `#store-category` text + toggles `.pending` class if `vendor.status === 'pending'` , and injects a verified-badge SVG into `#store-verified-badge` only if `vendor.status === 'verified'` .
- **#store-message-link** ("Message Vendor") — its `href` is set programmatically to `chat.html?chat=${vendorKey}` (`store.html:394-395`) so it deep-links straight into that vendor's conversation on `chat.html` .
- **#store-call-btn** (phone icon button) — click handler at `store.html:397-400` just calls `alert('Calling ${vendor.name} — hook this up to your call flow.')` . Not a real call.
- **#store-back-btn** (chevron "back" button) — `store.html:402-410` . If `document.referrer` exists and shares the page's origin, calls `window.history.back()` ; otherwise redirects to `explore.html` .
- **Products grid** (`#storeProductsGrid`) — rendered from `vendor.products` array via a `.map().join('')` template (`store.html:416-423`); each card's "Add to cart" button (`.add-btn`) is caught by the shared delegated `wireAddToCartButtons()` in `customer/assets/interactions.js` and routes to `cart.html` .
- **Star rating input** (`#star-input` `.star-btn` , 5 buttons) — `setStars(value)` (`store.html:470-475`) sets `starInput.dataset.value` and toggles `.active` on every button whose `data-star` is `<=` the clicked value (so

clicking star 3 highlights 1-2-3). Local DOM state only (`starInput.dataset.value`), reset to `0` after each successful submission.

- **#review-form submit ("Submit review")** — `store.html:481-507` . Reads `rating = Number(starInput.dataset.value)` and `text = document.getElementById('review-text').value.trim()` . Validation: `if (!rating) alert('Please select a star rating before submitting.')` and returns; `if (!text) { textEl.focus(); return; }` (empty text silently blocks submit, no message). On success: `reviews.unshift({ name: 'You', rating, date: new Date().toISOString().slice(0,10), text })` — prepends to the in-memory `reviews` array (a `.slice()` copy of `vendor.reviews` , `store.html:430`), then re-renders both `renderReviewSummary()` (average score + star count, `store.html:432-447`) and `renderReviewList()` (`store.html:449-462` , every review — including the new one — tagged "Verified purchase" regardless of any real purchase check, per the page's own comment that this can't actually verify an order). Resets the star input and clears the textarea. State is **entirely in-memory** (the `reviews` array closure variable) — reloading the page loses the new review.

customer/cart.html

All behavior in `customer/assets/cart.js` , loaded alongside `interactions.js` / `support.js` . Two static `.cart-item` rows ship in the HTML with no real cart backing them.

- **Quantity steppers (`.qty-btn` "-" / "+" , per cart item)** — `wireQuantitySteppers()` (`cart.js:22-44`). On page load, for each `.cart-item` , reads `initialQty` from the `.cart-item-actions` span text and `initialTotal` from `.cart-price` text (via `parseNaira()` , `cart.js:66-68` , strips non-digits), derives `unitPrice = initialTotal / initialQty` . On click: increments/decrements the displayed qty (min `1` , `Math.max(1, qty - 1)`), sets `.cart-price` text to `formatNaira(unitPrice * qty)` (`cart.js:70-73` , ₦ + `toLocaleString('en-NG')`), then calls `updateOrderSummary()` .
- **updateOrderSummary()** (`cart.js:46-64`) — sums every `.cart-price` element's parsed value into `subtotal` , reads the current delivery fee from the summary card's 2nd `.summary-row strong` (unchanged, just re-read), and writes `subtotal` into the 1st row's `<strong>` and `subtotal + delivery` into the 3rd (`.total`) row's `<strong>` . This directly targets `.summary-card .summary-row` by array index (`rows[0]` , `rows[1]` , `rows[2]`), so reordering the summary rows in HTML would break it.
- **.contact-vendor-btn (per cart item, "Contact vendor")** — `wireContactVendorButtons()` (`cart.js:80-86`); navigates to `chat.html` (the generic list, not a specific conversation — cart items don't carry a vendor id that maps to chat.html's mock conversation ids; this is called out as a known gap in the code comment and in DOCUMENTATION.md §8).
- **Delivery-option radios (`.delivery-options input[type=radio]` , per item)** — plain radio inputs, no JS handler; purely visual selection state, doesn't affect price/summary.
- **.summary-card .secondary-btn ("Save for later")** — `wireSaveForLaterButton()` (`cart.js:89-96`); click just calls `alert('Cart saved for later (hook this up to your save-for-later API).')` . No real save action.
- **.summary-card .primary-btn ("Checkout")** — `wireCheckoutButton()` (`cart.js:99-116`). Guards against double-click via `if (btn.disabled) return` . On click: disables button, sets text to "Processing...", then after `600ms` `setTimeout: alert('Order placed! (hook this up to your checkout/payment API)')` , re-enables/restores the button, and redirects to `dashboard.html` . No real order/payment is created.

customer/orders.html

Loads `../admin/assets/data.js` directly (the **only** place in the customer app that touches the admin `localStorage` state, key `vetra_admin_state_v1`), plus page-specific `customer/assets/orders.js` and `customer/assets/report-issue.js` .

- **Status filter tabs (`#order-filter-tabs .filter-tab[data-filter]`)** — `wireOrderFilterTabs()` (`customer/assets/orders.js:14-34`). Click sets `.active` on the clicked tab only, then shows/hides each `.order-card` in `#order-list` by comparing `filter` to `card.dataset.status` (values: `all` , `pending` , `processing` , `shipped` , `out-for-delivery` , `completed` , `cancelled`) — non-matching cards get `style.display = 'none'` .
- **Order card summary (`.order-card-summary` , `click-to-expand`)** — `wireOrderExpand()` (`orders.js:36-49`), delegated on `#order-list` . Click toggles `.open` on the parent `.order-card` (independent per-card accordion — multiple can be open at once) and sets `aria-expanded` on the summary button. Reveals `.order-track-panel`'s step timeline via CSS keyed off `.open` .
- **"Report an issue" (`[data-action="report-issue"]` , per order card)** — `customer/assets/report-issue.js` . Delegated click on `#order-list` (`report-issue.js:47-52`) finds the closest `.order-card` and calls `openModal(card)` (`report-issue.js:31-39`): sets `#report-order-id` / `#report-vendor-name` text from the card's

- `data-order-id / data-vendor-name`, resets `#report-form`, un-hides `#report-modal`, locks page scroll (`document.body.style.overflow = 'hidden'`), focuses `#report-reason`.
- Close paths** — `#report-modal-close`, `#report-cancel-btn`, clicking the overlay backdrop, or `Escape` — all call `closeModal()` (`report-issue.js:41-45`).
- #report-form submit ("Submit report")** — `report-issue.js:63-94`. Validation is native HTML5 `required` on `#report-reason` (a `<select>`) and `#report-details` (a `<textarea>`) — no custom regex. On submit: builds `reason = "${reasonSelect.value} (${orderId}): ${detailsInput.value.trim()}"`, then (if `VetraAdmin` is defined, i.e. the admin script loaded) calls `VetraAdmin.addReport({ type: 'vendor', targetId: vendorId, targetName: vendorName, reporter: CURRENT_BUYER, reason })` (`admin/assets/data.js:361`), where `CURRENT_BUYER = "Amaka Obi"` is hard-coded (`report-issue.js:20`) standing in for a real signed-in session. This writes into the same `localStorage`-backed state `admin/reports.html` reads. Each order card's `data-vendor-id` maps to a real admin vendor id only for `Naija Home Essentials` → `v3`; the other three order cards use synthetic ids (`jennet-jeans`, `gods-favour`, `me-n-u-chops`) that don't match any admin vendor record, so a report against them lands in the queue but can't deep-link to a real vendor-detail page.
- After submit, the triggering `[data-action="report-issue"]` button is replaced in the DOM with a `<span class="report-filed-tag">Report filed</span>` (`report-issue.js:85-90`), preventing the same order from being reported twice (this state is DOM-only — reloading the page restores the original button since nothing is persisted client-side for the order card itself, though the report itself *does* persist in admin's `localStorage`).
- "Contact vendor"** (per order's track panel) — plain `<a href="chat.html">`, not JS, opens the chat list (no specific vendor thread).

customer/chat.html

Entirely inline `<script>` at the bottom of the page; mock conversation data lives in a `chatData` array (`chat.html:185-240`), 4 vendors (`gods-favour`, `daniel-ice-fish`, `jennet-jeans`, `me-n-u-chops`), each with `status`: `'verified' | 'pending'`, `online`, `unread`, `preview`, `time`, and a `messages` array.

- Chat section tabs (`#chatTabs` .`chat-tab[data-section]` , "Pending"/"Verified")** — rendered by `renderChatList()` (`chat.html:255-317`); click sets `selectedSection` (module-level variable) and re-renders. Only chats whose `status` matches `selectedSection` are shown, further filtered by the search box's text.
- #chatSearch (search input)** — `input` event calls `renderChatList(e.target.value)` (`chat.html:504-507`); filters `chatData` by `chat.name / chat.preview` case-insensitively containing the query.
- Chat list item (.`chat-item` , per conversation)** — click calls `selectChat(chatId)` (`chat.html:311-316` , `319-395`). Sets `activeChatId`, toggles `.active` class on the clicked item, and — if mobile width (`window.innerWidth <= 900`) — adds `.mobile-chat-open` to `.chat-shell` (`updateMobileView()` , `chat.html:247-253`). Rebuilds `#chatWindow`'s entire innerHTML with that chat's header, message thread, and input bar.
- .`chat-back` (mobile back arrow, inside chat header)** — calls `selectChat(null)`, which clears `activeChatId` and restores the "Select a chat" empty-state markup.
- .`chat-close` (X button, inside chat header)** — same as `.chat-back`, calls `selectChat(null)`.
- .`chat-call` ("Call" button)** — `chat.html:406-431`. Guards against double-click (`if (callBtn.classList.contains('calling')) return`), adds a `.calling / aria-busy` state, builds a hidden `<a href="tel:${chat.phone} || '+0000000000'">` and programmatically `.click()` s it to trigger the OS dialer, then after `1400ms` removes the calling state and the temp element. Falls back to `alert('Calling ${phone}')` in a `catch` if the `tel:` trick throws.
- .`attach-btn` (📎 icon) + hidden .`file-input`** — click on `.attach-btn` calls `fileInput.click()`; the input's `change` event (`chat.html:440-450`) reads `fileInput.files`, and renders a `<span class="attachment-chip">📎 name (+N more)</span>` into `.attachment-preview`. Files are **never actually sent or uploaded** — purely a local filename preview.
- .`send-btn` (➤) and pressing Enter in the message input** — both call `sendMessage()` (`chat.html:459-482`). Trims the input value; if non-empty, pushes `{ role: 'outgoing', text, time }` onto that chat object's own `messages` array (mutating the in-memory `chatData` array directly — this is the "sent" message, kept only in JS memory, lost on reload), updates `chat.preview / chat.time` (so the list preview reflects the latest message), appends the message bubble's HTML directly into `.chat-body` and scrolls to bottom, clears the input, and re-runs `renderChatList()` so the sidebar preview/time update immediately.
- Deep link via `?chat=<id>` query param** — on `DOMContentLoaded` (`chat.html:495-511`), if the param matches a `chatData` entry's `id`, that chat's `status` becomes the initially `selectedSection` and `selectChat()` is called with it immediately — this is how `store.html`'s "Message Vendor" button and `orders.html`'s conversation links (where present) open a specific thread instead of the empty state.

customer/notifications.html

- **"Order delivered" card** — the only notification wrapped in an `<a href="orders.html">` (with `unread` class); plain navigation, no JS, but note it's the one entry point into `orders.html` outside Settings.
- **"Mark all read" (.page-link button)** — no `id`, no JS handler anywhere in the codebase; inert.
- Other notification cards ("Flash sale", "Payment confirmed") are plain non-interactive `<div>` s.

customer/settings.html

All interaction logic is an inline `<script>` at the bottom of the page (no separate JS file); also loads `../password-toggle.js` for the 3 password fields in the Security card.

- **#profile-edit-toggle ("Edit", Profile card)** — `enterEditMode()` (`settings.html:462-468`). Calls `fillProfileForm()` to copy the in-memory `profile` object (`fullName`, `phone`, `email`, `address`, hard-coded at `settings.html:429-434` — "Amaka Obi" etc., the same mock identity `report-issue.js` uses as `CURRENT_BUYER`) into the form inputs, hides `#profile-view` (adds `.hidden`), shows `#profile-form`, hides the edit-toggle button itself, focuses `#full-name`.
- **#profile-cancel (inside the edit form)** — `exitEditMode()` (`settings.html:470-474`); re-hides the form, re-shows the read-only view and the edit-toggle button, without saving anything.
- **#profile-form submit ("Save changes")** — `settings.html:481-490`. No validation (no `required` /regex on these inputs). Writes the 4 input values back onto the `profile` object, calls `renderProfileView()` to re-populate `#view-full-name` / `#view-phone` / `#view-email` / `#view-address`, then `exitEditMode()`. State is a plain JS object in the inline script's closure — resets to the hard-coded values on reload.
- **#password-edit-toggle ("Edit", Security card)** — `settings.html:512-515`. Calls `setPasswordEditable(true)` (`settings.html:504-510`), which clears `readOnly` on all 3 password inputs (`#current-password`, `#new-password`, `#confirm-password`) and enables `#password-submit-btn` (previously `disabled`); focuses the first field.
- **#password-cancel-btn** — `settings.html:517-520`; calls `form.reset()` then `setPasswordEditable(false)` (re-locks fields, disables submit, re-shows the edit-toggle button).
- **#password-form submit ("Update password")** — `settings.html:522-527`. No validation of match/strength/length despite being a password-change form. On submit: `e.preventDefault()`, `form.reset()`, `setPasswordEditable(false)` — no actual password update occurs (`// TODO: wire up to real update-password endpoint`).
- **#manage-payment-btn ("Manage payment methods")** — `settings.html:531-536`; click → `alert('Hook this up to your payment methods flow.')`.
- **#deactivate-account-btn ("Deactivate account")** — `settings.html:540-550`. Uses native `confirm('Deactivate your account? ...')`; if confirmed, `alert('Account deactivation requested (hook this up to your API.')`. No real deactivation.
- **"View orders →" link** (Account overview card) — plain `<a href="orders.html">`, not JS.
- **"Log out"** — plain `<a href="../signin.html">`, not JS (no session to actually clear).
- **Notification toggles (.toggle-switch input[type=checkbox], 3 rows)** — plain checkboxes, no JS handler; purely visual on/off state that doesn't persist or do anything.
- **Password reveal toggles** — injected by `../password-toggle.js` on all 3 password fields, same mechanism as `signin.html`.

6. Vendor (seller) app

All pages live in `vendor/` and share the sidebar: **Dashboard, Products, Orders, Earnings, Profile**.

Page	What it does
<code>dashboard.html</code>	Store KPIs (revenue, orders, active listings, store views), a recent-orders list, and a "My Products" preview grid. Has the Add Product button, which opens a modal form.
<code>products.html</code>	Full product grid with Edit / Remove actions per card (<code>vendor/assets/products.js</code>).
<code>orders.html</code>	Order list with status filter tabs (Pending/Processing/Shipped/Out for delivery/Delivered/Cancelled) via <code>wireOrderFilterTabs</code> in <code>orders.js</code> . Each order has an Update shipment button opening a modal (<code>assets/order-tracking.js</code>) to set shipment status, carrier, and tracking number — updates that

Page	What it does
	order row's status pill and tracking line in place. Below the order list, a Reports against your store panel (read-only — see below) shows reports filed against this vendor.
earnings.html	Revenue breakdown / earnings history, plus a Payout Account section (see below) above the payout history list.
profile.html	Store profile form (name, owner, email, phone, address, bio — starts read-only, "Edit Profile" unlocks the fields), a Business Verification (KYC) section (see below), notification toggles, security section (change password, 2FA, sign out), and a Danger Zone (deactivate store / delete account).
notifications.html	Vendor-side notification feed.

Add Product modal ([vendor/assets/add-product.js](#)): a full form — name, category, price, a stock quantity stepper, description, up to 4 image upload slots with live preview, and one optional video upload slot — that builds a new product card and prepends it to the grid on submit ([handleSubmit](#) → [buildProductCard](#)). This is in-memory only: reloading the page loses any product you added, because (like the customer app) nothing here persists to [localStorage](#) .

Reports against your store ([orders.html](#) , [vendor/assets/reports.js](#)). A vendor can see reports filed against their store but can't resolve or dismiss them — that stays admin-only ([admin/reports.html](#)), keeping one place where a report is actually closed out. What a vendor *can* do is respond: an open report's **Submit evidence** button opens a modal (a response textarea plus a mocked photo-upload slot borrowed from the Add Product modal's media-upload UI) — submitting it swaps the button for an "Evidence submitted — awaiting review" tag on that card. This is deliberately a read-only mirror of the same report data admin sees, not a second copy of the moderation queue.

Business Verification / KYC ([profile.html](#) , [vendor/assets/kyc.js](#)). Closes a gap where the site's buyer-facing copy claimed "every Vendor passes ID and business verification before their first listing ever goes live" ([buyer-protection.html](#)) with no actual UI for a vendor to do that verification. The form takes a CAC registration number plus two uploads — a valid ID and a CAC certificate (image preview for photos, a filename chip for a PDF) — reusing the Add Product modal's upload-slot pattern. Submitting requires all three fields, then locks the form and flips the status badge to "Pending review." Like the rest of this app, it's a page-local mock with no persistence — reloading resets it, and it does **not** write into the admin console's [localStorage](#) . The admin-side review ([admin/vendor-detail.html](#) 's "Business Verification (KYC)" card, described below) is separate seed data representing documents an admin has already reviewed, not something this form actually submits to.

Payout Account ([earnings.html](#) , [vendor/assets/payout.js](#)). Closes a gap where [vendor-protection.html](#) describes payouts running "straight to the bank account on file" and this same page already captioned every payout-history row "Payout to bank account" — with no UI anywhere to actually set one, on either the vendor form or the admin data model. The new section (above Payout History) takes a bank, a 10-digit account number, and an account name; saving validates the account number's length, then swaps to a read-only view showing the account masked ([.... 6789](#)) with an **Edit Account** button. Page-local mock, like KYC — nothing persists past a reload, and (deliberately, for a real version) even this demo never shows the full account number again once saved, only the masked form, modeling what a real implementation's display rule should be even though the underlying mock data is trivial to inspect.

Shared JS: [assets/interactions.js](#) ([VetraUI](#) , sidebar toggle/collapse only — no banner carousel or add-to-cart delegation, since the vendor app has neither banners nor a public product grid) and [assets/support.js](#) (identical Smartsupp open/close logic to the customer copy, but targets [#header-support-btn](#) on vendor pages, which carries the [.support-btn](#) class the same delegated listener matches).

Shared reference: - [#header-sidebar-toggle](#) / [#app-sidebar-reopen](#) / [#sidebar-collapse-toggle](#) — [VetraUI.wireSidebarToggle\(\)](#) / [wireSidebarCollapse\(\)](#) ([vendor/assets/interactions.js:15-44](#)), functionally identical to the customer version. - [#header-support-btn](#) (header icon) — opens Smartsupp via the same [openLiveChat\(\)](#) / [ensureCloseButton\(\)](#) pattern in [vendor/assets/support.js:68-80](#) .

[vendor/dashboard.html](#)

- [#add-product-btn](#) ("Add Product") — [wireAddProductButton\(\)](#) in [vendor/assets/product-actions.js:34-40](#) ; calls [window.VetraAddProduct.open\(\)](#) ([vendor/assets/add-product.js:228-234](#)), which stores [document.activeElement](#) for focus restoration, un-hides [#add-product-modal](#) , locks body scroll, focuses [#ap-name](#) .
- **Add Product modal** — see full breakdown under "Add Product modal (shared)" below.

- **Product grid actions** (`.edit-btn` / `.remove-btn` , delegated on `.vendor-products-grid`) — `wireVendorProductActions()` (`product-actions.js:13-32`). Reads the product name from the closest `.vendor-product-card .product-name` . `edit-btn` → `alert('Edit "${name}" — hook this up to your edit-product page.')` (no real edit page exists). `remove-btn` → native `confirm('Remove "${name}" from your store?')` ; if confirmed, `card.remove()` — removes the DOM node only (no backend delete, so it reappears on reload).
- **"See all" links** (Recent Orders → `orders.html` , My Products → `products.html`) — plain `<a>` , not JS.

vendor/products.html

- **#add-product-btn** — identical wiring to dashboard.html (`product-actions.js` + `add-product.js` , same modal markup duplicated in this page's HTML).
- **.edit-btn / .remove-btn** — identical `wireVendorProductActions()` behavior as dashboard.html, operating on this page's own `.vendor-products-grid` (9 static product cards).

Add Product modal (shared markup/JS across dashboard.html and products.html) — `vendor/assets/add-product.js` , module `VetraAddProduct` : - **Form fields**: `#ap-name` (text, `required`), `#ap-category` (`<select>`, `required`, one of 8 fixed categories), `#ap-price` (`number`, `min="0"` `step="1"`, `required`), `#ap-stock` (`number`, `min="0"` `step="1"`, `required`, default `0`), `#ap-description` (`<textarea>`, `required`). - **Stock stepper** (`#ap-stock-decrement` / `#ap-stock-increment`) — `wireStockStepper()` (`add-product.js:120-133`); `step(delta)` clamps to `Math.max(min, current + delta)` where `min = Number(stockInput.min) || 0`. - **Image upload slots (4x, #ap-image-grid .media-upload-slot[data-slot-index])** — `wireImageSlots()` (`add-product.js:108-118`); each slot's hidden `<input type="file">` `change` event calls `setImagePreview(index, file)` (`add-product.js:42-87`), which creates an object URL (`URL.createObjectURL(file)`), injects an `<img class="media-preview-img">` into the slot, adds a per-slot remove (`×`) button that clears the file and revokes the object URL, and toggles `.has-media` on the slot. - **Video upload slot (#ap-video-slot / #ap-video-input / #ap-video-remove)** — `wireVideoSlot()` (`add-product.js:135-155`); same object-URL preview pattern via `setVideoPreview(file)` (`add-product.js:89-106`), shows the filename in `.video-file-name`. - **Form submit ("Add Product")** — `handleSubmit(e)` (`add-product.js:196-226`). Calls native `form.reportValidity()` first (enforces all `required` fields + number min/step) and returns early if invalid. Then checks `imageFiles.some(Boolean)` — if no image was ever attached, scrolls to the first image slot and `alert('Add at least one product photo before saving.')`, blocking submission (this check exists *in addition to* the native validity check, since the file inputs themselves aren't marked `required`). On success: builds a new card via `buildProductCard({ name, price, stock, imageUrl })` (`add-product.js:176-194`) — `imageUrl` is a **fresh** `URL.createObjectURL()` call on the first attached image file (kept independent of the form's own preview URLs, which get revoked on close) — HTML-escapes `name` via `escapeHtml()` (`add-product.js:30-37`) to prevent injection, computes the stock-pill via `stockPillMarkup(stock)` (`add-product.js:170-174` , "low" styling when `stock === 0 || stock <= 5`), then `grid.prepend(card)` onto whichever `.vendor-products-grid` exists on the current page, and scrolls the new card into view. Finally calls `close()`. - `close()` (`add-product.js:236-242`) — hides the modal, unlocks scroll, calls `resetForm()` (clears all inputs/previews and revokes every object URL), restores focus to `lastFocusedEl`. - **Close paths**: `#add-product-close`, `#add-product-cancel`, clicking the overlay backdrop, or `Escape` — all call `close()`. - **State**: entirely in-memory (closure variables `imageFiles`, `videoFile`, etc., and the DOM card itself) — a newly added product disappears on page reload, exactly like Edit/Remove.

vendor/orders.html

Loads `assets/orders.js`, `assets/order-tracking.js`, `assets/reports.js` in that order (no shared "orders" state object between them beyond the DOM itself).

- **Status filter tabs (#order-filter-tabs .filter-tab[data-filter])** — `wireOrderFilterTabs()` (`vendor/assets/orders.js:10-30`), functionally identical to the customer orders.js version but targets `.order-item` rows inside `#order-list` (not `.order-card`) and compares against `row.dataset.status`.
- **"Update shipment" (.order-manage-btn , per order row)** — `vendor/assets/order-tracking.js`. Delegated click on `#order-list` (`order-tracking.js:55-59`) finds the closest `.order-item` and calls `openModal(row)` (`order-tracking.js:39-47`): sets `#shipment-order-id` from the row's `.order-id` text, pre-fills `#shipment-status` from `row.dataset.status` (default `"pending"`), `#shipment-carrier` / `#shipment-tracking` from `row.dataset.carrier` / `row.dataset.tracking`, un-hides `#shipment-modal`, locks scroll.
- **#shipment-form submit ("Save update")** — `order-tracking.js:71-108`. No required-field validation (carrier/tracking are optional per the form's own label). On submit: writes `status` / `carrier` / `tracking` back onto `activeRow.dataset.*`, updates the row's `.status-pill` class to `status-pill ${status}` and its text via the `STATUS_LABEL` map (`order-tracking.js:21-28` — note `completed` displays as **"delivered"**, matching the customer-facing wording), and either creates or removes a `.order-tracking-line` element depending on whether

`carrier` / `tracking` are set (inserted right before `.order-manage-btn` via `insertBefore`). All 6 status values map 1:1 with customer/orders.js's filter tabs (`pending`, `processing`, `shipped`, `out-for-delivery`, `completed`, `cancelled`).

- **Close paths:** `#shipment-modal-close`, `#shipment-cancel-btn`, backdrop click, `Escape` — call `closeModal()` (`order-tracking.js:49-53`).
- **State:** DOM-only (`row.dataset.*` + the row's own child elements) — resets on reload, no cross-page sync back to `customer/orders.js`'s mock data.
- **"Submit evidence"** (`[data-action="evidence"]`), per open report card in the "Reports against your store" panel — `vendor/assets/reports.js`. Delegated click on `#report-list` (`reports.js:37-42`) finds the closest `.report-card` and opens `#evidence-modal` (`reports.js:23-29`), clearing/focusing `#evidence-text`.
- **#evidence-form submit ("Submit")** — `reports.js:53-66`. `#evidence-text` textarea is `required` (native validation only — no length/content check). On submit, replaces that card's entire `.report-actions` innerHTML with `<span class="badge dismissed evidence-sent-tag">Evidence submitted – awaiting review</span>` — a vendor cannot resolve/dismiss the report itself (that stays admin-only via `admin/reports.html`); this is purely a "vendor responded" marker, DOM-only, resets on reload.
- **Close paths:** `#evidence-modal-close`, `#evidence-cancel-btn`, backdrop click, `Escape` — `closeModal()` (`reports.js:31-35`).
- Report data itself (`#report-list .report-card`, e.g. `rep-1`, `rep-2`) is static seed markup in `orders.html`, **not** pulled from `admin/assets/data.js`'s `localStorage` state — this panel is a page-local mock mirror, unlike `customer/orders.html`'s report-filing which does write into admin's real state.

vendor/earnings.html

`vendor/assets/payout.js` — Payout Account section only; the stat cards and Payout History list are static, non-interactive.

- **#payout-account-form submit ("Save Payout Account")** — `payout.js:50-61`. Fields: `#payout-bank` (`<select>`, `required`, one of 9 banks/ `Other`), `#payout-account-number` (`text`, `maxlength="10"`, `inputmode="numeric"`, `required`), `#payout-account-name` (`text`, `required`). **Validation:** `/^\d{10}$/.test(numberInput.value.trim())` — must be exactly 10 digits (a NUBAN); on failure, sets `numberInput.style.borderColor = '#e0475c'` and focuses it, blocking submit. On success, calls `showSavedView()` (`payout.js:31-42`): populates `#payout-view-bank` / `#payout-view-number` (masked via `maskAccountNumber()`, `payout.js:27-29` → `"...." + number.slice(-4)`, so the plaintext number is **never re-displayed** once saved, though it is briefly present in the `numberInput.value` until the form is hidden) / `#payout-view-name`, changes `#payout-status-badge` text to "Account on file" and swaps its class from `.status-not-set` to `.status-set`, hides the form and shows `#payout-account-view`.
- **#payout-edit-btn ("Edit Account", shown once an account is saved)** — `payout.js:63`, calls `showForm()` (`payout.js:44-48`): hides the saved view, re-shows the form, and un-hides `#payout-cancel-btn` **only if** the badge is not `.status-not-set` (i.e., cancel only appears once an account already exists, so a first-time setup can't be "cancelled" back to an undefined state).
- **#payout-cancel-btn** — `payout.js:65-67`, calls `showSavedView()` directly (re-shows whatever was last saved, discarding in-progress edits — note it does **not** call `form.reset()`, so the form fields retain the edited-but-uncommitted values if reopened).
- **State:** DOM-only via the module-level `badge` / `view` / form element references — nothing persists past a reload; the account number, once saved, is not stored anywhere real (per the file's own header comment, this demo intentionally still holds the plaintext briefly in the input to allow the masked-view demo, but never re-displays it).

vendor/profile.html

Two JS files: `vendor/assets/profile.js` (summary card, store-details form, security, danger zone, sign-out) and `vendor/assets/kyc.js` (Business Verification section).

- **#avatar-edit-btn (small pencil badge on the avatar)** — `wireAvatarEditButton()` (`profile.js:12-19`); click → `alert('Hook this up to your avatar upload flow.')`. No real upload.
- **#profile-edit-btn ("Edit Profile", header shortcut)** — `wireProfileEditButton()` (`profile.js:36-46`); calls `setProfileFormEditable(true)` (`profile.js:26-33`, clears `readOnly` on every `.form-input` / `.form-textarea` inside `#profile-form` and adds `.is-editing` to the form), focuses `#store-name`, and smooth-scrolls the form into view. All 6 fields (`#store-name`, `#owner-name`, `#store-email`, `#store-phone`, `#store-address`, `#store-bio`) ship with the `readonly` HTML attribute by default, so nothing is editable until this button is clicked.

- **#profile-form submit ("Save Changes")** — `wireProfileForm()` (`profile.js:49-76`). No validation logic beyond native (none of the fields carry `required` /pattern). On submit: reads `#store-name` / `#owner-name` values (trimmed) and, if non-empty, writes them back onto the summary card's `#profile-store-name` text and `#profile-owner-name` (interpolated as `"${ownerName} · Vendor since Jan 2026"` — the "Jan 2026" join date is hard-coded, not derived from any real data), calls `setProfileFormEditable(false)` to re-lock the form, then `alert('Profile saved (hook this up to your save API).')`.
- **#profile-cancel-btn** — `profile.js:69-75`; calls `form.reset()` (reverts to the HTML's original `value` attributes) then re-locks via `setProfileFormEditable(false)`.
- **Business Verification (KYC) form (#kyc-form)** — `vendor/assets/kyc.js`. Fields: `#kyc-cac-number` (text, `required`), two upload slots (`#kyc-id-slot` / `#kyc-id-input`, `#kyc-cac-slot` / `#kyc-cac-input`, accepting `image/*` and `image/*,.pdf` respectively). Each slot's `change` event calls `setPreview(key, file)` (`kyc.js:34-81`) — same object-URL preview pattern as `add-product.js`, except a non-image file (e.g. a PDF CAC certificate) renders a `.media-file-name` chip with the filename instead of an `<img>` (`kyc.js:57-64`), since PDFs can't be previewed as an image.
- **#kyc-form submit ("Submit for verification")** — `kyc.js:99-119`. **Validation:** `cacNumberInput.value.trim()` non-empty and both `slots.id.file` and `slots.cac.file` must be set — all three or nothing; failure sets `#kyc-note` text to `"Enter your CAC number and upload both documents before submitting."` in red (`#b91c1c`) and returns, without alerting or blocking via native validity (the file inputs aren't marked `required` in HTML, so this JS check is the only enforcement). On success: sets `#kyc-status-badge` text to "Pending review" and swaps its class from `.status-not-submitted` to `.status-pending`, clears the note's color and sets it to `"Submitted – Vetra usually reviews new documents within 24 hours."`, then calls `setLocked(true)` (`kyc.js:90-97`) which makes `#kyc-cac-number` `readOnly`, disables both file inputs, adds `.is-locked` to both slots, and hides the submit button (`submitBtn.hidden = true` — there is no unlock/edit path after submission within this page).
- **State:** page-local, in-memory only; does **not** write to `localStorage` or reach into the admin console's data at all (explicitly the opposite convention from `customer/orders.js` 's report-filing) — reload fully resets the KYC section back to "Not submitted".
- **#change-password-btn** — `wireChangePasswordButton()` (`profile.js:79-86`); → `alert('Hook this up to your change-password flow.')`.
- **#deactivate-store-btn** — `wireDangerZoneButtons()` (`profile.js:89-101`); native `confirm('Deactivate your store? ...')`, then on confirm `alert('Store deactivated (hook this up to your API).')`.
- **#delete-account-btn** — same function (`profile.js:103-112`); `confirm('Delete your vendor account? This cannot be undone.')`, then `alert('Account deletion requested (hook this up to your API).')`.
- **#sign-out-btn** — `wireSignOutButton()` (`profile.js:116-125`); `confirm('Sign out of your vendor account?')`, and if confirmed, `window.location.href = '../signin.html'` (an actual redirect, unlike the other danger-zone buttons which just alert).
- **Notification toggle switches (`.switch input[type=checkbox]`, 4 rows) and 2FA toggle** — plain checkboxes, no JS handler, purely visual.

vendor/notifications.html

- **"Mark all read" (`.page-link` button)** — no `id` /handler in the codebase; inert, same as the customer equivalent.
- All 4 notification cards are static, non-interactive `<div>` s (no links, no click handlers) — this is the one notifications page across both apps where even the "top" notification isn't wrapped in a navigable link.
- Only shared sidebar/support/header behavior applies (`vendor/assets/interactions.js`, `vendor/assets/support.js`).

7. Admin console — new in this session

Why it was built

The request was for a place where VETRA staff can monitor customers and vendors platform-wide and suspend accounts, plus "other things that should be on the admin section." The console lives entirely under `admin/`, reuses the exact same app-shell/design system as the vendor app (so it feels native to the site, not bolted on), and — unlike the rest of the prototype — **persists its state to `localStorage`** under the key `vetra_admin_state_v1`, because moderation actions (suspending someone, resolving a report) need to survive a page reload to feel real. `admin/assets/data.js` is the single place that owns

this state; every admin page reads and writes through its functions (`VetraAdmin.*`) instead of touching `localStorage` directly, so swapping in a real backend later means changing one file.

Getting in — and "who's logged in"

`admin/login.html` — a staff-only sign-in screen, styled like `signin.html` but with no Buyer/Vendor toggle. It's linked from the bottom of `signin.html` ("Vetra staff? Admin sign in"). Like the rest of the site's auth, it doesn't check a real password — but it does do one real thing: it looks the email you typed up against the admin team roster in `admin/assets/data.js` and, if it matches, tells the data layer **which admin is now "logged in"** (`VetraAdmin.setCurrentAdminByEmail()`). That simulated session is what makes the rest of this section possible — every action taken anywhere in the console is attributed to whoever is currently "logged in," and the console's own role check (Super Admin vs. Moderator vs. Support) reads from it too. The sign-in page lists the three seeded demo accounts and their roles so you can try each one. See §9 for what a real version needs (this is still just an email lookup, not a password check).

Passwords, photos, and "who did what"

Three things were added on top of the original monitor/suspend request, all in direct response to follow-up asks in this session:

- **Reset Password (customers & vendors).** Every customer/vendor row (on `customers.html` / `vendors.html`) and every detail page has a **Reset Password** button. Confirming it generates a temporary password and shows it to the admin in a "share this with them securely" panel — there's no real account system to actually change a password against, so this is as far as a backend-less prototype can go, but the action is logged to that account's activity history like any other.
- **Admin profile photos.** Settings → Admin Profile has a real, working **Change Photo** upload (not a placeholder): it reads the chosen image with `FileReader` , stores it as a base64 data URL on that admin's team record, and immediately reflects it in the profile card, the header avatar on every admin page, and the Admin Team list. This works because the admin console already persists to `localStorage` , unlike the rest of the prototype.
- **Attribution — "which admin attended to a case."** Every activity log entry now records who performed it (`actorId` / `actorName`), and every report gets an `attendedBy` / `attendedAt` pair the moment it's resolved, dismissed, or resolved-via-suspend. That attribution shows up everywhere the relevant record appears — the activity feed ("by Adaeze Umeh"), a report card ("Attended by Adaeze Umeh"), and a customer/vendor's own activity history — regardless of which admin is viewing it. Case-level attribution is intentionally visible to every admin; it's the console-wide activity *browsing* below that's role-gated.

Pages

`dashboard.html` — **Platform overview.** Six KPI cards (Total Customers, Total Vendors, Platform Orders, Platform Revenue, Suspended Accounts, Open Reports), a **Quick Actions** row linking straight into a pre-filtered view of the other pages, a **Recent Activity** preview (last 6 audit-log entries), and a **Pending Vendor Applications** table where a new store's application can be **Approved** or **Rejected** right from the dashboard.

`customers.html` — **Manage buyers.** A searchable, filterable table of every customer (name/email, join date, order count, lifetime spend, status). Each row has **View** and either **Suspend** or **Reactivate** depending on current status. Suspending asks for an optional reason via a confirm modal — the reason is stored in the activity log so there's a record of *why*.

`customer-detail.html?id=<id>` — **One customer, in full.** Clicking a customer's name or **View** opens their complete profile instead of a quick-look popup: contact info (phone, delivery address), how and when they signed up, last login, lifetime order/spend stats, **any reports filed against their account** (pulled from the Reports queue), and their **entire activity history** — every suspend/reactivate action ever taken on this specific account, not just the platform-wide feed. Suspend/Reactivate works from here too, so a moderator reviewing an account doesn't have to bounce back to the list to act on what they just read.

`vendors.html` — **Manage stores.** Same shape as Customers, plus a **Pending Approval** state: new vendor applications show **Approve/Reject** instead of Suspend until an admin acts on them. Filter tabs: All / Active / Pending Approval / Suspended.

`vendor-detail.html?id=<id>` — **One store, in full.** Same idea as the customer detail page: store contact info, category, a short description, owner, join date, last login, products/orders/revenue stats, any reports filed against the store, and its complete activity history. The action buttons adapt to status — **Approve/Reject** while pending, **Suspend/Reactivate** once live. A **Business Verification (KYC)** card shows the vendor's submitted CAC number and ID/CAC document filenames (`admin/assets/data.js` 's per-vendor `kyc` object — status one of `not_submitted` / `pending` / `verified` / `rejected`); a `pending` submission gets **Verify Documents/Reject** actions through the shared confirm modal (`VetraAdmin.setVendorKycStatus()`), logged to that vendor's activity history like any other action. This is independent of the

account-level Approve/Reject status — a vendor can in principle be `active` with KYC still `pending`, since the two aren't currently forced to move together (see §8 for why).

reports.html — **Reports & disputes**. A moderation queue of flagged accounts, listings, and buyer↔vendor disputes (e.g. "item didn't match description," a chargeback dispute, a suspicious listing). Each open report can be **Marked Resolved**, **Dismissed**, or — if it's against a customer or vendor — resolved with a one-click **Suspend Account**, which suspends the account *and* marks the report resolved in a single confirmation.

activity.html — **Activity log / audit trail**. Every suspend, reactivate, approve, reject, resolve, dismiss, password reset, invite, and admin-removal action performed anywhere in the admin console is appended here automatically, newest first, with a relative timestamp ("2h ago"), a type tag, and — for anything an admin actually did — **who did it** ("by Adaeze Umeh"). Filterable by Account Actions / Vendor Approvals / Reports / Orders / Logins. This is what makes the console feel like a real moderation tool rather than a set of disconnected switches — every destructive action leaves a trace.

Visibility here depends on role: a **Super Admin** sees everyone's activity and gets a **"filter by admin"** dropdown to narrow it to one person; a **Moderator or Support** admin only sees platform events (orders, customer/vendor logins — nothing admin-specific) plus their *own* actions, with a note explaining that Super Admins can see more. This is enforced by `VetraAdmin.getVisibleActivity()`, which every activity view (this page and the dashboard's preview) calls instead of the raw log — but, like every other role check in this prototype, it's a client-side filter, not a real permission boundary (see §9).

settings.html — **Admin profile & platform controls**. - **My Profile**: reflects whoever actually signed in (name, email, role badge), with a working **Change Photo** upload. - **Platform Controls**: toggles for "require approval for new vendors," "auto-flag suspicious listings," "allow guest checkout" (**on by default** — buyers can check out without an account unless an admin turns it off here), "maintenance mode" (cosmetic switches — see §9). - **Admin Team**: the full list of who has console access and their role (Super Admin / Moderator / Support), each with their photo if they've set one, and a **Remove** action that goes through the shared confirm modal. Removing the platform's *last* Super Admin is blocked with an explanation, so the console can't be locked with zero full-access admins. - **Add Admin — invite + email verification**. Clicking **Add Admin** doesn't add someone immediately: it collects name/email/role and creates a *pending invitation*, then opens a **Verify Email** step. Since this prototype has no mail server to actually deliver anything, the code that "would have been emailed" is shown right in the UI, and — per how this was asked for — **entering anything in the verification field completes it**; this is a simulated verification step to demonstrate the invite→verify shape of a real onboarding flow, not a real code check (see `VetraAdmin.verifyTeamInvite()` in `admin/assets/data.js`, and §9). Pending invitations appear in their own list with **Verify/Cancel** actions until completed. - **Notifications** and **Security** sections matching the vendor profile page's pattern (password, 2FA, sign out). - **Data & Danger Zone → Reset Demo Data**: wipes the admin console's `localStorage` state and reloads the original seed data (team roster and any pending invites included). This exists *specifically because* this is a prototype with no real backend — it's how you get back to a clean demo state after clicking around, and it would not exist in a production build.

The shared confirm modal

Every destructive or state-changing action (suspend, approve, reject, resolve, dismiss, reset demo data) goes through **one reusable confirm dialog** (`AdminUI.confirm()` in `admin/assets/ui.js`), not a bare `window.confirm()`. It shows exactly what's about to happen, names the target by name, optionally collects a reason, and only then calls into `VetraAdmin.*` to make the change. This was a deliberate choice: an admin console where "Suspend" fires immediately on click, with no confirmation, is a foot-gun.

What "add other things that should be on the admin section" became

Beyond the explicitly requested monitoring + suspend features, this session added: 1. **Vendor approval workflow** (pending → active/rejected) — without this, every new vendor signup would go live unmoderated. 2. **Reports & disputes queue** — a place for the "someone flagged this" workflow that any marketplace needs; suspending straight from a report closes the loop between "a problem was reported" and "an admin acted on it." 3. **Activity log / audit trail** — accountability for who did what, which also makes the console testable/demoable (you can see the effect of your own actions). 4. **Admin team/roles list** — a console with a hardcoded single admin user doesn't reflect that real platforms have multiple staff with different permission levels. 5. **Reset Demo Data** — a prototype-specific affordance so the demo doesn't get "stuck" in a suspended/modified state after someone tries the buttons. 6. **Admin login gate** (`admin/login.html`) — the original ask assumed an admin section existed on the site already reachable somehow; since it didn't, this is the entry point, reachable from `signin.html`. 7. **Full customer/vendor detail pages** (`customer-detail.html`, `vendor-detail.html`) — requested directly: monitoring "all their account activity and user data" needed more room than a quick-look popup could give, so each account now gets its own page with full profile info, a filtered activity history, and any reports against it. 8. **Add/Remove Admin** on Settings — also requested directly: the console needed a real way to grow (or shrink) its own staff list, not just display three hardcoded names. 9. **Password reset for**

customers and vendors — requested directly: an admin console that can suspend an account but can't help someone locked out of it is missing a basic support tool. 10. **Admin profile photos** — requested directly, with a real (client-side) upload rather than a placeholder, since the console already has somewhere to persist it. 11. **Per-admin activity attribution + role-gated activity browsing** — requested directly ("let super admins see the activities of other admins... let all admins see which admin attended to a case"): every log entry and every report now records who acted on it, visible to all; but *browsing* the full log is Super-Admin-only, with everyone else scoped to platform events and their own actions. 12. **Admin invite + simulated email verification** — requested directly: adding a new admin now goes through an invite-and-verify step instead of adding them outright, standing in for the real "confirm your email" step a production onboarding flow would have. 13. **Guest checkout, on by default** — requested directly: `admin/settings.html` 's "Allow guest checkout" toggle now starts checked, and `signin.html` gained a "Continue as Guest →" link (buyer tab only) straight into `customer/dashboard.html`, so the platform-control setting has a matching entry point on the actual sign-in page.

Full interaction reference (every button, exact behavior)

The page-by-page tour above says what each page is for; this is the granular version — every interactive control, the function that handles it, and where its state actually lives. All of it is driven through `VetraAdmin.*` (`admin/assets/data.js`) and `AdminUI.*` (`admin/assets/ui.js`), so those two files are named throughout rather than repeated per page.

Shared across every admin page (`admin/assets/ui.js`): - The header's sidebar-collapse icon (`#header-sidebar-toggle`) toggles `.hidden-desktop` on `#app-sidebar` and shows a floating reopen button (`#app-sidebar-reopen`) — `wireSidebarToggle()` . - The sidebar's own collapse arrow (`#sidebar-collapse-toggle`) toggles a `.collapsed` class (icon-only rail) — `wireSidebarCollapse()` . - Every header avatar image (`.header-avatar img`) is kept in sync with whichever admin is "logged in" — `applyCurrentAdminAvatar()` , called on every page load and again after a photo upload. - **The shared confirm modal** (`#confirm-modal` , `AdminUI.confirm({title, bodyHtml, confirmLabel, danger, showReason, onConfirm})`) is what every suspend/approve/reject/resolve/dismiss/remove action across the whole console routes through — no page wires its own modal. `danger: true` turns the confirm button red; `showReason: true` reveals an optional textarea whose value is passed to `onConfirm(reason)` . If a page's markup doesn't include `#confirm-modal` at all, `confirm()` silently falls back to a native `window.confirm()` so it never hard-fails. - **The shared info modal** (`AdminUI.info({title, bodyHtml, confirmLabel})`) is the same overlay in a one-button mode, used only to show a value back to the admin (a generated temp password, "admin added" confirmation) — no `onConfirm` , no reason field. - **The shared activity-feed renderer** (`AdminUI.renderActivityFeed(container, entries)`) is what `dashboard.html` 's preview, `activity.html` 's full list, and both detail pages' "Account Activity"/"Activity History" sections all call — one function, so a change to how an entry renders (its icon, its "by X" attribution) updates everywhere at once.

dashboard.html (`admin/assets/dashboard.js`): - Six KPI cards are pure display, computed once from `VetraAdmin.getStats()` on load (`renderStats()`) — no interaction. - The "Pending Vendor Applications" table (`renderPendingVendors()` , capped at 4 rows) reads `VetraAdmin.getVendors().filter(v => v.status === "pending")` . Its **Approve** button opens a confirm dialog ("Their store and listings will go live immediately") whose `onConfirm` calls `VetraAdmin.setVendorStatus(id, "active")` , then re-renders the pending table, stats, and activity feed. **Reject** opens a `danger: true, showReason: true` confirm ("They can re-apply later") calling `setVendorStatus(id, "rejected", reason)` . - The Recent Activity preview shows `VetraAdmin.getVisibleActivity().slice(0, 6)` — role-scoped (see the Activity Log entry below), not the raw log.

customers.html (`admin/assets/customers.js`): - The search input (`#customer-search` , `input` event) and status dropdown (`#customer-status-filter` , `change` event) both call one `render()` that filters `VetraAdmin.getCustomers()` by `name / email` substring match (lowercased) and by exact `status` , then rebuilds the table body from scratch every time — there's no diffing, the whole `<tbody>` is replaced. - The status filter's initial value can come from the URL: `customers.html?filter=suspended` (used by `dashboard.html` 's "View Suspended Accounts" quick action) pre-selects that filter on load. - Each row's name/avatar cell and its **View** button both link to `customer-detail.html?id=<id>` — same destination, two click targets. - **Reset Password** opens a confirm dialog; on confirm, `VetraAdmin.resetCustomerPassword(id)` generates a temporary password (returned as a plain string — see `admin/assets/data.js`) and `AdminUI.info()` displays it in a `.reveal-panel` for the admin to copy and relay manually. This is the prototype's stand-in for "email a reset link" — see DOCUMENTATION.md §9 for why a real backend must never show this value to the admin. - **Suspend** (`danger: true, showReason: true`) calls `VetraAdmin.setCustomerStatus(id, "suspended", reason)` ; **Reactivate** (shown instead, once suspended) calls the same function with `"active"` and no reason.

vendors.html (`admin/assets/vendors.js`): - Same search-input pattern as customers, plus **filter tabs** (`#vendor-filter-tabs` , `filter-tab` , click-to-activate) instead of a dropdown: All / Active / Pending Approval / Suspended. `?filter=pending` in

the URL pre-activates a tab the same way `customers.html` 's query param does. - `render()` always excludes `status === "rejected"` vendors from every view, even "All" — a rejected application isn't shown anywhere in this list (it still exists in `VetraAdmin.getVendors()` , just filtered out here). - A **pending** row shows **Approve/Reject** instead of Suspend; both behave exactly as `dashboard.html` 's versions (same confirm copy, same `setVendorStatus()` calls) — this file and `dashboard.js` each have their own copy of that logic rather than sharing one function, which is a small, acknowledged duplication (same pattern noted for `customer/assets/interactions.js` vs `vendor/assets/interactions.js` in DOCUMENTATION.md §8). - **Reset Password** behaves identically to the customers page, via `VetraAdmin.resetVendorPassword(id)` .

`customer-detail.html?id=<id>` (`admin/assets/customer-detail.js`): - On load, reads `id` from the URL and calls `VetraAdmin.getCustomer(id)` ; if it returns nothing (bad/missing id, or the customer was removed by a demo-data reset), the page swaps to a `#customer-not-found` empty state and never calls `render()` . - `render()` populates every field (name, email, signup method, phone, address, joined date, last login with a relative "time ago" via `VetraAdmin.timeAgo()`) directly by element id, then calls four sub-renderers: `renderStats()` (orders/spend/status tiles), `renderActions()` (Reset Password + Suspend-or-Reactivate, same confirm/action pattern as the list view, but note the button click handler here is attached with `wrap.querySelector('[data-action="suspend"]', [data-action="activate"]')` — a single selector matching whichever one actually rendered, since only one of the two exists in the DOM at a time), `renderReports()` (see below), `renderActivity()` . - **Reports filed against this account:** `VetraAdmin.getReportsForTarget("customer", id)` . If empty, the entire section is `hidden = true` rather than showing an empty state — a customer with no reports shows no reports section at all, not a "no reports" message. - **Activity History:** `VetraAdmin.getActivityForTarget("customer", id)` — every log entry for this specific account, not the platform-wide feed, rendered through the same `AdminUI.renderActivityFeed()` used everywhere else.

`vendor-detail.html?id=<id>` (`admin/assets/vendor-detail.js`): - Same not-found/render split as customer-detail. `renderActions()` branches three ways instead of two: `pending` → Approve/Reject, `suspended` → Reactivate, else → Suspend — matching `vendors.html` 's three-state list view. - **Business Verification (KYC)** (`renderKyc()` , added when the KYC feature was built): reads `vendor.kyc` (`{status, cacNumber, idDocumentName, cacDocumentName, submittedAt, reviewedAt}`). `not_submitted` shows a plain "nothing to review" message with no actions. `pending` shows the CAC number and both document names (via `docChip()` , which renders "Not uploaded" in muted text if a filename is missing) plus **Verify Documents** (`VetraAdmin.setVendorKycStatus(id, "verified")`) and **Reject** (`danger: true, showReason: true` , → `setVendorKycStatus(id, "rejected", reason)`) buttons. `verified` / `rejected` show the same fields plus a "Reviewed \<date>" line and no action buttons — a decision, once made, isn't reversible from this screen. - Reports/Activity sections work exactly like the customer-detail page, scoped to `("vendor", id)` instead of `("customer", id)` .

`reports.html` (`admin/assets/reports.js`): - Filter tabs (`#report-filter-tabs` , All/Open/Resolved/Dismissed) work the same click-to-activate pattern as the vendors page; `?filter=open` etc. in the URL pre-selects one. - Every report card shows **Mark Resolved** and **Dismiss** when `status === "open"` ; a report against a customer or vendor (not a product) additionally shows **Suspend Account**. Nothing shows once a report is resolved/dismissed — a decided report has no further actions here. - **Mark Resolved** → confirm → `VetraAdmin.setReportStatus(id, "resolved")` . **Dismiss** → confirm → `setReportStatus(id, "dismissed")` . - **Suspend Account** is the one three-way action on this page: its confirm dialog is `danger: true, showReason: true` with the label "Suspend & resolve," and its `onConfirm` calls **both** `VetraAdmin.setVendorStatus()` / `setCustomerStatus()` (whichever `report.type` is) **and** `VetraAdmin.setReportStatus(id, "resolved")` in the same click — suspending the account and closing the report out in one confirmation, rather than requiring two separate actions for what's conceptually one decision.

`activity.html` (`admin/assets/activity.js`): - Filter tabs narrow by `type` (Account/Vendor/Report/Order/Login/All) — pure client-side array filter on whatever `VetraAdmin.getVisibleActivity()` already returned. - **The one page whose behavior literally differs by who's logged in:** if `VetraAdmin.getCurrentAdmin().role === "Super Admin"` , an admin-filter dropdown (`#activity-admin-filter-wrap`) appears, populated from `VetraAdmin.getTeam()` , letting a Super Admin narrow the feed to one person's actions (`entries.filter(a => a.actorId === adminFilter)` , layered on top of the type filter). Anyone else instead sees a static note (`#activity-scope-note`) explaining they're seeing platform events plus their own actions only — and their query never even reaches for other admins' actions, since `getVisibleActivity()` itself is already scoped (see `admin/assets/data.js` below).

`settings.html` (`admin/assets/settings.js`) — the most interaction-dense page: - **My Profile:** `renderMyProfile()` shows whoever `VetraAdmin.getCurrentAdmin()` currently resolves to. **Change Photo** (`wireAvatarUpload()`) is a real, working upload: clicking the button forwards the click to a hidden `<input type="file">` , then a `FileReader` reads the chosen image as a base64 data URL and calls `VetraAdmin.setTeamMemberAvatar(me.id, dataUrl)` — which persists it to that admin's `localStorage` record, re-renders the profile card and team list, and calls `AdminUI.applyCurrentAdminAvatar()` so the header avatar updates immediately without a reload. - **Admin Team** (`renderTeam()`): each row's **Remove** button opens a `danger: true` confirm; on confirm, `VetraAdmin.removeTeamMember(id)` either succeeds (re-renders the list) or returns

`{ok: false, error: "last-super-admin"}`, in which case an `AdminUI.info()` dialog explains the block instead of removing anyone — the UI never lets the console reach zero Super Admins. - **Add Admin** is a two-step flow, not one form: 1. `wireAddAdminModal()` — the **Add Admin** button opens `#add-admin-modal` (name/email/role fields). Submitting calls `VetraAdmin.inviteTeamMember({name, email, role})`, which creates a pending-invite record with a freshly generated verification code, closes the invite modal, re-renders the pending-invites list, and immediately opens the verify modal for the invite that was just created. 2. `openVerifyModal(inviteId)` / `wireVerifyInviteModal()` — the verify modal just asks for a code (any non-empty value is accepted, per how this was explicitly requested — see DOCUMENTATION.md §7). Submitting calls `VetraAdmin.verifyTeamInvite(inviteId, codeEntered)`; on success it closes the modal, re-renders both the team list and pending-invites list, and shows an `AdminUI.info()` "Admin added" confirmation naming the new member's role. - **Pending Invitations** (`renderPendingInvites()`, section hidden entirely when the list is empty): each row's **Verify** button re-opens the same verify modal as step 2 above; **Cancel** opens a `danger: true` confirm calling `VetraAdmin.cancelInvite(id)`. - **Reset Demo Data**: a `danger: true` confirm warning that "any changes you've made in this browser will be lost," calling `VetraAdmin.resetDemoData()` (wipes the `localStorage` key and re-seeds it) and then hard-navigating to `dashboard.html`. - **Sign Out**: a plain confirm, no state change beyond navigating to `./signin.html` — it doesn't actually clear `currentAdminId`, so reopening any admin page in the same browser still resolves to the same "logged in" admin (a known limitation of simulating a session with a single `localStorage` field instead of a real one — see DOCUMENTATION.md §9). - Platform-control toggles (require vendor approval, auto-flag listings, guest checkout, maintenance mode) have no JS behind them at all — flipping one changes nothing else on the page or in `localStorage`; they're purely cosmetic, matching the equivalent switches on `vendor/profile.html`.

`login.html` — no dedicated JS file; the sign-in logic is a `<script>` block inline in the page itself. Clicking **Sign in to console** validates both fields are non-empty (turns the field's border red if not, via inline `style.borderColor`), then calls `VetraAdmin.setCurrentAdminByEmail(emailField.value.trim())` — a case-insensitive lookup against `admin/assets/data.js`'s seeded team roster by email only, **no password comparison happens at all**, matching the "any password works" note printed right on the page. If the email doesn't match any team member, `setCurrentAdminByEmail()` returns `null` and the "current admin" simply doesn't change — the page still redirects to `dashboard.html` regardless, where everything renders based on whichever admin was last successfully resolved (possibly from a previous session), not a fresh "no one is logged in" state. There's a 250ms `setTimeout` before the redirect purely for a "Signing in..." button-state effect, not for anything asynchronous actually happening.

Where the state actually lives (`admin/assets/data.js`)

Everything above eventually bottoms out in one `const VetraAdmin = (() => {...})();` module, backed by a single `localStorage` key, `vetra_admin_state_v1`. A few implementation details worth knowing if you're extending this file:

- `load()` reads that key on first script execution; if it's missing/unparseable (private browsing, first visit, cleared storage), it deep-clones the `SEED` constant and persists that instead — so the app never renders with truly empty state. It also backfills fields that didn't exist in older saved states (`pendingInvites`, `currentAdminId`, and — added with the KYC feature — a default `kyc: {status: "not_submitted", ...}` on any vendor record missing one), so a browser with state saved from before a feature existed doesn't crash reading `undefined.status`.
- **Every mutating function follows the same shape**: look up the record, mutate the in-memory `state` object directly (not immutably — no copies), call the module-private `save()` (which just `JSON.stringify`s the whole state back into `localStorage`), then call `logActivity()` with a human-readable message built from the record's own fields (never a generic "record updated"). This is why every action across the console is independently auditable — the log entry is written from inside the same function that made the change, not bolted on by whichever page called it.
- `logActivity(type, message, target, opts)`: `target` is `{type, id}` for anything with a detail page to link back to (`null` for pure login/system events). The fourth parameter, `opts.systemEvent`, is a later addition (added alongside the buyer-report-filing feature) that skips attributing the entry to `getCurrentAdmin()` — needed because a report filed from a customer's browser has no admin actor at all, and without this flag the log would wrongly credit whichever admin last signed in on that machine.
- `getVisibleActivity()` is the one function everything role-sensitive routes through instead of the raw `state.activity` array: a Super Admin gets it unfiltered; anyone else gets it filtered to `actorId === null || actorId === getCurrentAdmin().id`. This is a **client-side filter only** — see DOCUMENTATION.md §9 for why that's not a real permission boundary, and `backend/src/routes/admin.routes.js`'s `GET /api/admin/activity` for the server-side version that actually enforces it.
- `addReport()` (added for the buyer-report-filing feature) is the one function in this file explicitly designed to be called from *outside* the admin console — see `customer/assets/report-issue.js`, which loads this file directly via `<script src="./admin/assets/data.js">` rather than treating it as admin-only.

8. Site-wide review: redundancies, fixes, and what still needs attention

An audit pass was run across the public site, customer app, and vendor app (the admin section was excluded from the audit since it was just built and independently verified separately — every stat card, table, filter tab, search box, and confirm-modal flow on all six admin pages was clicked through in an automated browser test and confirmed to work, including that suspending a customer persists after a page reload). Everything below that was flagged as an actual bug was fixed in this session and re-verified with another automated browser pass (checked live: cart quantity/price/subtotal math, "Contact vendor" navigation, the checkout flow, the chat Send button, both settings buttons, the vendor product grid after the JS consolidation, and the signup→signin vendor-mode handoff — zero console errors).

Fixed

Where	Problem	Fix
<code>customer/cart.html</code>	The Checkout , Save for later , quantity −/+ steppers, and Contact vendor buttons had no JS behind them at all — clicking any of them did nothing. This is the site's core purchase flow, so it was the highest-impact finding.	Added <code>customer/assets/cart.js</code> : quantity steppers now recompute that line's price and the whole order summary (subtotal + total) live; "Contact vendor" opens the chat app; "Checkout"/"Save for later" give real click feedback (a processing state, then a confirmation) in place of the mock payment/save API this prototype doesn't have.
<code>customer/chat.html</code>	The Send button (and pressing Enter) did nothing — messages could never actually be sent in the demo chat UI.	Wired <code>sendMessage()</code> into the chat-window render: it appends the message to that conversation's history, renders it into the thread, updates the sidebar preview/time, and clears the input.
<code>customer/settings.html</code>	" Manage payment methods " and " Deactivate account " had no <code>id</code> and no handler — every other button on that page was wired, these two were an apparent oversight.	Gave both an id and a handler matching the same confirm+aler "hook this up to your API" placeholder pattern already used by <code>vendor/assets/profile.js</code> danger-zone buttons, so the whole site's mock-action convention now applies consistently here too.
<code>signup.js</code>	After validating the form, the Continue button just sat on "Creating account..." forever — the signup flow had no redirect at all, unlike sign-in.	Added the same delayed-redirect pattern <code>signin.js</code> uses, sending the new account to <code>signin.html</code> (or <code>signin.html#Vendor</code> if they registered as a vendor).
<code>signin.js</code>	Didn't handle a <code>#Vendor</code> URL hash, even though <code>signup.js</code> already produced/expected that pattern for its own hash-based deep link — a latent inconsistency that the signup fix above would otherwise have silently tripped over (landing on the Buyer tab after registering as a Vendor).	Added the matching hash check so <code>signin.html#Vendor</code> now pre-selects the Vendor tab, closing the gap between the two auth pages.
<code>vendor/assets/dashboard.js</code> +	Both files contained byte-for-byte identical <code>wireVendorProductActions()</code> / <code>wireAddProductButton()</code>	Merged both into one <code>vendor/assets/product-</code>

Where	Problem	Fix
<code>vendor/assets/products.js</code>	functions — accidental copy-paste, not the intentional shared-CSS convention used elsewhere, so a future edit to one copy could silently drift from the other.	<code>actions.js</code> , updated <code>dashboard.html</code> and <code>products.html</code> to load it, deleted the two duplicates. Re-verified Edit/Remove and the Ad Product modal still work on both pages.

Reviewed and left as-is (working as intended)

- **Sidebar/bottom-nav links** across every customer and vendor page resolve correctly and are consistent sibling-to-sibling.
- `vendor/assets/add-product.js` modal (shared by `dashboard.html` and `products.html`) opens/closes correctly via its close button, Cancel, backdrop click, and Escape.
- **Contact form, FAQ accordion** (`script.js`) on the public pages work.
- Several buttons intentionally `alert()` a "hook this up" placeholder rather than navigate (vendor Edit product, avatar edit, change password) — this is the established, deliberate pattern for "the backend doesn't exist yet" across the codebase, not a bug, so these were left alone.
- `customer/assets/interactions.js` vs `vendor/assets/interactions.js` and `signin.js` vs `signup.js` 's buyer/vendor toggle logic are near-duplicates of each other by the same intentional per-area convention described in §2/§6 — low-priority, cosmetic overlap, not touched.

Still open (needs a human decision, not a quick fix)

- `customer/cart.html` 's "Contact vendor" opens the chat list rather than a specific conversation, because cart line items don't currently carry a vendor id that maps to `chat.html` 's mock conversation data. Once products/cart items carry a real vendor id, point this at `chat.html?chat=<id>` the same way `store.html` already does.
- **Two separate, unreconciled fictional vendor lists.** `admin/assets/data.js` 's `SEED.vendors` (the vendors admin actually manages, `v1 – v7`) and `customer/store.html` 's own `VENDORS` object (`gods-favour`, `daniel-ice-fish`, `jennet-jeans`, `me-n-u-chops`, used across `store.html` and now `orders.html` 's report-filing) were built independently and only coincidentally overlap once (`Naija Home Essentials = v3`). A report filed against one of the other three shows up correctly in `admin/reports.html` 's main queue (see §5/§8) but won't deep-link to a real vendor-detail page, since its `targetId` doesn't match any admin vendor record. Fixing this properly means picking one vendor list and having both apps reference it — bigger than a quick patch, so flagged here rather than partially addressed.
- Everything in §9 below (auth, database, uploads, cart/chat persistence) — these are architecture-level gaps, not bugs in the existing code.

Feature gaps closed by a follow-up audit

A later audit pass (live-rendering check across 15 priority pages at mobile and desktop widths, plus a static sweep for orphaned images and dead JS) found zero rendering bugs but three gaps between what the site *claims* and what it actually lets someone do — all three closed in this session:

1. **No order history or delivery tracking**, despite "real-time delivery tracking" being advertised on `index.html` and `buyer-protection.html`, and vendors being told to "log tracking details" on `vendor-protection.html`. Added `customer/orders.html` (order history + expandable tracking timeline) and an **Update shipment** modal on `vendor/orders.html` (status/carrier/tracking-number, per order) — see §5 and §6 above.
2. **No customer-facing review UI**, despite "verified reviews only" being a repeated selling point. Added a Reviews section (summary + review list + write-a-review form) to `customer/store.html` — see §5 above.
3. **No vendor-facing view of reports filed against their store** — admin had a full moderation queue (`admin/reports.html`) but a vendor had no way to even see a report existed, let alone respond to it, despite `vendor-protection.html` walking vendors through exactly that flow. Added a read-only **Reports against your store** panel to `vendor/orders.html` — see §6 above.

The same audit also found 5 image files with zero references anywhere in the codebase (the original `logo.png` before the brand-asset generation pass, two unused mark-only logo variants, and two duplicate `avatar-dummy.jpg` files shadowed by an identically-purposed `.png`). Moved to `extras/` rather than deleted outright, in case they're wanted later.

A fourth gap of the same shape was reported directly by name afterward, not found by an audit pass:

1. **No vendor-facing KYC/business-verification UI**, despite `buyer-protection.html` stating "every Vendor passes ID and business verification before their first listing ever goes live." Added a **Business Verification (KYC)** section to `vendor/profile.html` (CAC number + ID/CAC certificate upload, `vendor/assets/kyc.js`) and a matching review card to `admin/vendor-detail.html` (`VetraAdmin.setVendorKycStatus()`) — see §6 and §7 above.

A follow-up audit, specifically hunting for this exact pattern (a claim in the site's copy with no matching feature anywhere), found two more — both reported and then closed in the same session:

1. **No way for a buyer to actually file a dispute/report against an order.** `buyer-protection.html` and `vendor-protection.html` both describe a detailed "buyer reports an issue → escrow stays held → vendor gets 48 hours to respond → admin rules" flow, and `admin/reports.html` exists to review exactly that — but `customer/orders.html`'s "Report an issue" button just opened generic live chat, and `admin/assets/data.js` had no `addReport()` function for anything to call at all. Added a **File a report** modal to `customer/orders.html` (`customer/assets/report-issue.js`) and a new `VetraAdmin.addReport()` — see §5 above for how this one deliberately reaches across into the admin console's `localStorage`, unlike every other customer/vendor feature.
2. **No vendor UI to set or view a payout bank account**, despite `vendor-protection.html` describing payouts running "straight to the bank account on file," and `vendor/earnings.html` itself captioning payout-history rows "Payout to bank account" as if one already existed — the field was missing from both the vendor form and the admin data model, not just missing a form. Added a **Payout Account** section to `vendor/earnings.html` (`vendor/assets/payout.js`) — see §6 above.

Neither is built yet — flagging them here so they're tracked the same way the four gaps above were, rather than only living in chat history.

9. What a real backend needs to replace

This prototype intentionally fakes several things. Before any of this touches real users:

1. **Authentication.** `signin.js`, `signup.js`, and `admin/login.html`'s inline script all just check "is this field non-empty" (`admin/login.html` additionally matches the email against the team roster to know *which* admin it is, but never checks a password) and redirect. There is no real password check, no session/token, and — critically — **no server-side check that a signed-in admin is actually an admin**. Anyone who knows the URL can open `admin/dashboard.html` directly and, via `localStorage`, even set `currentAdminId` to any admin's id. A real build needs real auth (e.g. sessions or JWTs) with role-based access control gating every `admin/*.html` route server-side, not just a client-side lookup.
2. **Admin roles gate almost nothing server-side.** The one real enforcement right now is client-side: `getVisibleActivity()` filters what a non-Super-Admin's activity log shows, and `removeTeamMember()` blocks deleting the last Super Admin. Nothing stops a Moderator or Support admin from calling `setVendorStatus`, `resetCustomerPassword`, etc. directly — the UI just doesn't currently offer every action to every role by design intent, not by enforcement. Once real auth exists, every `VetraAdmin.set*` / `reset*` / `add*` / `remove*` call needs a real permission check behind it.
3. **The admin invite/verification flow is fully simulated.** `inviteTeamMember()` generates a code and displays it directly in the UI instead of emailing it, and `verifyTeamInvite()` accepts *any* non-empty value as correct — there is no real email delivery or code matching. A real build needs an actual mail provider and a code comparison that can fail.
4. **Password reset doesn't reset anything real.** `resetCustomerPassword()` / `resetVendorPassword()` generate a temporary string and show it to the admin — there's no real customer/vendor password store to write it to, since buyer/vendor accounts aren't real accounts in this prototype either (see point 1). A real build needs this to actually invalidate the old password and email/notify the account holder, not display the new one to the admin.
5. **A real database.** Every "list of customers/vendors/products/orders" is hard-coded JS or, in the admin console, `localStorage` — which is per-browser and never seen by anyone else. All of it needs to move to a real database behind an API.
6. **admin/assets/data.js is the seam.** It already isolates every read/write behind named functions (`getCustomers`, `setCustomerStatus`, `resetCustomerPassword`, `getVendors`, `setVendorStatus`, `resetVendorPassword`, `getReports`, `setReportStatus`, `getActivity`, `getVisibleActivity`, `logActivity`, `getTeam`, `setTeamMemberAvatar`, `removeTeamMember`, `inviteTeamMember`, `verifyTeamInvite`, `getCurrentAdmin`, `setCurrentAdminByEmail`, `getStats`, plus the `getActivityForTarget` / `getReportsForTarget` lookups the detail

pages use). Swapping its internals from `localStorage` to `fetch()` calls against a real API — and its avatar storage from base64 data URLs to real file uploads — is the intended upgrade path; no other admin file talks to storage directly.

7. **Platform-control toggles** on `admin/settings.html` (require vendor approval, auto-flag listings, guest checkout, maintenance mode) are currently cosmetic switches with no effect — they need to be wired to real server-side flags that other parts of the app actually check.
8. **File uploads** (product images/video in the vendor "Add Product" modal, avatar edit buttons) only preview locally in the browser; nothing is actually uploaded anywhere.
9. **Cart and chat** (`customer/cart.html` , `customer/chat.html`) are UI-only — no real cart persists across pages, and no real messages are sent.

10. Design system reference (for anyone extending a page)

Shared building blocks used across customer, vendor, and admin (defined per-area in each `assets/style.css` , kept visually identical):

- **Layout:** `.app-shell` → `.sidebar` (desktop) + `.phone` (scroll container) → `header#app-header` → `main.page-shell` → `nav.bottom-nav` (mobile).
- **Cards/sections:** `.vendor-section` (generic card container, used well beyond vendor pages), `.stat-grid` / `.stat-card` (KPI tiles).
- **Tables (admin only):** `.table-wrap` + `table.data-table` , row actions via `.table-actions button` with semantic classes `.btn-view/.btn-suspend/.btn-activate/.btn-approve/.btn-reject` .
- **Status pills:** `.status-pill` (orders: pending/processing/shipped/out-for-delivery/completed/cancelled — the last two states added for order tracking), `.badge` (admin + vendor reports panel: active/suspended/pending/resolved/dismissed/open).
- **Forms:** `.form-grid` / `.form-group` / `.form-input` / `.form-textarea` , `.switch` (toggle), `.danger-zone` (destructive settings). `customer/assets/style.css` also defines a lighter `.field-group` (label + input/select/textarea) shared by `settings.html` and `store.html` 's review form.
- **Modals:** `.modal-overlay` + `.modal-panel` (Add Product, confirm dialogs, detail views, vendor/orders.html's Update Shipment and Submit Evidence modals). `customer/assets/style.css` gained its own copy of this block for `orders.html` 's File a Report modal — same class names as vendor/admin, kept in its own file per the usual convention.
- **Filter tabs:** `.order-filter-tabs` `.filter-tab.active` .
- **Report cards:** `.report-list` / `.report-card` / `.report-card-head` / `.report-reason` / `.report-meta-row` / `.report-actions` — originally admin-only (`admin/assets/style.css`), the same class names/shapes now also exist in `vendor/assets/style.css` for the read-only vendor-side view, so a report renders identically in both places.

If you add a new page to any app, copy the closest existing sibling page's `<head>` /sidebar/header/bottom-nav markup rather than writing it from scratch — this is how every existing page in the codebase was built, and it's what keeps the three apps visually consistent.